

Tutorial de Programação em MIPS

Prof. Dr. Clodoaldo Aparecido de Moraes Lima

2026

Objetivos do Tutorial

- Introduzir a arquitetura MIPS
- Compreender registradores e memória
- Implementar algoritmos em Assembly MIPS
- Relacionar código C com Assembly
- Manipular pilha, funções e arquivos
- Trabalhar com imagens e filtros

Introdução à Arquitetura MIPS

A Arquitetura de Computadores MIPS é uma das arquiteturas mais utilizadas no ensino de organização e arquitetura de computadores. Seu projeto segue o paradigma RISC (Reduced Instruction Set Computer), cuja principal característica é o uso de um conjunto reduzido de instruções simples e eficientes.

O objetivo da arquitetura MIPS é executar instruções de forma rápida, previsível e organizada, facilitando tanto a implementação do hardware quanto a compreensão do funcionamento interno de um processador.

Ao estudar MIPS, o aluno compreende como programas em linguagens de alto nível, como C, são traduzidos para instruções de máquina executadas diretamente pelo processador.

Arquitetura MIPS

- Arquitetura RISC
- Instruções simples e rápidas
- Uso intensivo de registradores
- Separação entre memória e CPU

A arquitetura MIPS pertence à família RISC. Nesse modelo:

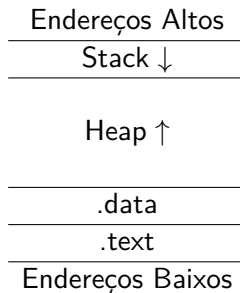
- As instruções possuem formato simples
- A maioria das instruções executa em um único ciclo
- O acesso à memória é separado das operações aritméticas
- O uso de registradores é intensivo
- O hardware torna-se mais simples e eficiente

Diferente das arquiteturas CISC (Complex Instruction Set Computer), o MIPS evita instruções muito complexas e prioriza operações pequenas e rápidas.

Segmentos de memória no MIPS

Segmento	Função
.text	Armazena as instruções do programa (código Assembly executável). O processador busca as instruções deste segmento durante a execução.
.data	Armazena variáveis globais, vetores, strings e dados estáticos definidos pelo programador.
heap	Região utilizada para alocação dinâmica de memória. Usada por estruturas criadas em tempo de execução, como vetores dinâmicos e listas encadeadas.
stack	Região utilizada para chamadas de função, variáveis locais, salvamento de registradores e recursão. A pilha cresce de endereços maiores para menores.

Exemplo visual da memória



Registradores Importantes

Registrador	Função	Descrição
\$zero	constante 0	Sempre possui o valor 0. Muito utilizado em operações aritméticas e comparações. Não pode ser alterado.
\$v0	retorno/syscall	Armazena valores retornados por funções e também o código das syscalls utilizadas pelo sistema.
\$a0-\$a3	argumentos	Registradores usados para passagem de parâmetros para funções. Permitem enviar até 4 argumentos diretamente.
\$t0-\$t9	temporários	Registradores temporários utilizados em cálculos intermediários. Não são preservados entre chamadas de função.

Registradores Importantes

Registrador	Função	Descrição
\$s0-\$s7	salvos	Registradores usados para armazenar valores importantes que devem ser preservados entre chamadas de função.
\$sp	stack pointer	Aponta para o topo da pilha (stack). Utilizado em chamadas de função, recursão e armazenamento temporário.
\$ra	return address	Armazena o endereço de retorno de uma função chamada com a instrução jal.

Explicação dos registradores

- **\$zero**: facilita operações lógicas e comparações, pois sempre contém zero.
- **\$v0**: além de guardar retornos de funções, define qual syscall será executada.
- **\$a0-\$a3**: usados para transferência rápida de parâmetros entre funções.
- **\$t0-\$t9**: ideais para variáveis temporárias em expressões matemáticas e laços.
- **\$s0-\$s7**: utilizados quando um valor precisa sobreviver após chamadas de funções.
- **\$sp**: controla a pilha de execução do programa.
- **\$ra**: garante que o programa consiga retornar corretamente após executar uma função.

Primeiro Programa

```
1 .data
2 msg: .asciiz "Ola Mundo!\n"
3
4 .text
5 .globl main
6 main:
7     li $v0, 4
8     la $a0, msg
9     syscall
10
11     li $v0, 10
12     syscall
```

Syscalls Importantes

Código	Syscall	Argumentos	Retorno
1	print_int	\$a0 = inteiro	–
2	print_float	\$f12 = float	–
4	print_string	\$a0 = endereço da string	–
5	read_int	–	\$v0 = inteiro lido
6	read_float	–	\$f0 = float lido
8	read_string	Buffer em \$a0 e tamanho máximo em \$a1	String armazenada no buffer
10	exit	–	encerra programa
11	print_char	\$a0 = caractere ASCII	–
12	read_char	–	\$v0 = caractere lido

Observações:

- Syscalls são serviços fornecidos pelo simulador MARS/SPIM.
- O código da operação deve ser colocado em \$v0.
- Os argumentos normalmente são passados em \$a0-\$a3.
- Resultados retornam em \$v0 ou registradores de ponto flutuante.

Exemplo de Uso de Syscalls

```
1 .data
2 msg: .asciiz "Digite um numero: "
3
4 .text
5 main:
6     # imprime string
7     li $v0, 4
8     la $a0, msg
9     syscall
10
11    # le inteiro
12    li $v0, 5
13    syscall
14
15    # valor lido fica em $v0
16    move $t0, $v0
17
18    # imprime inteiro
19    li $v0, 1
20    move $a0, $t0
21    syscall
22
23    # encerra
24    li $v0, 10
25    syscall
```

Leitura de Inteiro

```
1 li $v0, 5
2 syscall
3 move $t0, $v0
```

O valor lido fica em \$v0.

Estruturas de Repetição em MIPS

- Utilizadas para executar instruções repetidamente
- Equivalentes aos laços:
 - for
 - while
 - do-while
- Em MIPS os laços são implementados usando:
 - labels
 - desvios condicionais
 - saltos

Principais instruções:

Instrução	Função
beq	branch if equal
bne	branch if not equal
j	salto incondicional
slt	set less than
addi	soma imediata

Exemplo em Linguagem C

```
1 #include <stdio.h>
2
3 int main(){
4
5     int i;
6
7     for(i = 0; i < 10; i++){
8         printf("%d\n", i);
9     }
10
11     return 0;
12 }
```

Objetivo: Imprimir os números de 0 até 9.

Tradução para MIPS

Código C

```
1
2 int i;
3
4 for(i = 0; i < 10; i++){
5     printf("%d\n", i);
6 }
7
8 }
```

Código MIPS

```
1
2 .text
3 .globl main
4
5 main:
6     li $t0, 0
7     li $t1, 10
8
9
10 loop:
11
12     beq $t0, $t1, fim
13
14     li $v0, 1
15     move $a0, $t0
16     syscall
17
18     li $v0, 11
19     li $a0, 10
20     syscall
21
22     addi $t0, $t0, 1
23
24     j loop
25
26 fim:
27
28     li $v0, 10
29     syscall
```

Mapeamento C $\rightarrow$ MIPS

C	MIPS
<code>int i = 0</code>	<code>li \$t0, 0</code>
<code>i < 10</code>	<code>beq \$t0, \$t1, fim</code>
<code>i++</code>	<code>addi \$t0, \$t0, 1</code>
<code>printf</code>	<code>syscall</code>
<code>for</code>	<code>loop + jump</code>

Observação:

Em Assembly não existem estruturas prontas como `for` ou `while`. O programador implementa o controle usando labels e desvios.

Estrutura de Repetição – while

Código C

```
1
2 #include <stdio.h>
3
4 int main(){
5
6     int i = 0;
7
8     while(i < 5){
9
10         printf("%d\n", i);
11
12         i++;
13     }
14
15     return 0;
16 }
```

Lógica:

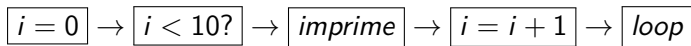
$$i < 5$$

Enquanto a condição for verdadeira, o loop continua executando.

Código MIPS

```
1
2 .text
3 .globl main
4
5 main:
6
7     # i = 0
8     li $t0, 0
9     # limite = 5
10    li $t1, 5
11 while:
12    # se i == 5 -> fim
13    beq $t0, $t1, fim
14    # imprime i
15    li $v0, 1
16    move $a0, $t0
17    syscall
18    # imprime '\n'
19    li $v0, 11
20    li $a0, 10
21    syscall
22    # i++
23    addi $t0, $t0, 1
24    # volta ao while
25    j while
26 fim:
27    li $v0, 10
28    syscall
```

Fluxo do Loop



Execução:

$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 9$

Estrutura Condicional – if else

Código C

```
1
2 #include <stdio.h>
3
4 int main(){
5
6     int x;
7
8     scanf("%d", &x);
9
10    if(x >= 0){
11
12        printf("Positivo\n");
13
14    }else{
15
16        printf("Negativo\n");
17
18    }
19
20    return 0;
21 }
```

Lógica:

$$x \geq 0$$

● verdadeiro → positivo

Código MIPS

```
1 .data
2 msgPos:
3 .asciiz "Positivo\n"
4 msgNeg:
5 .asciiz "Negativo\n"
6
7 .text
8 .globl main
9 main:
10     # le inteiro
11     li $v0, 5
12     syscall
13     move $t0, $v0
14     # testa x < 0
15     bltz $t0, else
16 if:
17     # imprime positivo
18     li $v0, 4
19     la $a0, msgPos
20     syscall
21     j fim
22 else:
23     # imprime negativo
24     li $v0, 4
25     la $a0, msgNeg
26     syscall
27 fim:
28     li $v0, 10
29     syscall
```

Endereçamento de Vetores

Como acessar elementos de um vetor em MIPS Para acessar um vetor de inteiros:

$$\text{endereco} = \text{base} + i \times 4$$

Onde:

- **base**: endereço inicial do vetor na memória
- **i**: índice do elemento desejado
- **4**: quantidade de bytes ocupados por um inteiro

Exemplo de vetor

10	20	30	40
----	----	----	----

Supondo:

$$\text{base} = 1000$$

$$\text{vet}[0] = 1000$$

$$\text{vet}[1] = 1004$$

$$\text{vet}[2] = 1008$$

$$\text{vet}[3] = 1012$$

Cada posição ocupa 4 bytes.

Organização do Vetor na Memória

Endereço	Elemento	Valor
1000	<i>vet</i> [0]	10
1004	<i>vet</i> [1]	20
1008	<i>vet</i> [2]	30
1012	<i>vet</i> [3]	40

Observação:

- O processador não entende diretamente o índice.
- O acesso é feito através de endereços de memória.
- Por isso é necessário calcular:

$$i \times 4$$

Exemplo em C

```
1  
2 int vet[4] = {10,20,30,40};  
3  
4 int x = vet[2];
```

O que acontece?

$$\text{endereço} = \text{base} + 2 \times 4$$

$$\text{endereço} = 1000 + 8$$

$$\text{endereço} = 1008$$

Logo:

$$x = 30$$

Tradução para MIPS

Código C

```
1
2 int vet[4] =
3 {10,20,30,40};
4
5 int x;
6
7 x = vet[2];
```

Código MIPS

```
1
2 .data
3
4 vet:
5 .word 10,20,30,40
6
7 .text
8
9 la $t0, vet
10
11 # i = 2
12 li $t1, 2
13
14 # i * 4
15 sll $t2, $t1, 2
16
17 # endereco
18 add $t3, $t0, $t2
19
20 # vet[2]
21 lw $t4, 0($t3)
```

Explicação

- la carrega o endereço base
- sll multiplica por 4
- add calcula o endereço final
- lw lê o valor da memória

Manipulação Matricial – Parte 1

Código em C

```
1
2 #include <stdio.h>
3
4 int main(){
5
6     int mat[3][3] = {
7         {1,2,3},
8         {4,5,6},
9         {7,8,9}
10    };
11
12    int i,j;
13
14    for(i=0;i<3;i++){
15
16        for(j=0;j<3;j++){
17
18            printf("%d ",
19                mat[i][j]);
20
21        }
22
23        printf("\n");
24    }
25
26    return 0;
27 }
```

Código MIPS – Parte 1

```
1
2 .data
3
4 mat:
5     .word 1,2,3
6     .word 4,5,6
7     .word 7,8,9
8
9 esp:
10     .asciiz " "
11
12 nl:
13     .asciiz "\n"
14
15 .text
16 .globl main
17
18 main:
19     li $t0, 0
20
21 for_i:
22     beq $t0, 3, fim
23     li $t1, 0
24
25 for_j:
26     beq $t1, 3, prox
27     mul $t2, $t0, 3
28     add $t2, $t2, $t1
```

Manipulação Matricial – Parte 2

Descrição do Algoritmo

- Dois laços percorrem a matriz
- O endereço é calculado por:

$$((i \times \text{colunas}) + j) \times 4$$

- O valor é carregado com:
lw
- A impressão usa syscalls

Registadores

\$t0	linha
\$t1	coluna
\$t2	offset
\$t3	endereço
\$t4	valor

Código MIPS – Parte 2

```
1      sll $t2, $t2, 2
2      la $t3, mat
3      add $t3, $t3, $t2
4      lw $t4, 0($t3)
5      li $v0, 1
6      move $a0, $t4
7      syscall
8      li $v0, 4
9      la $a0, esp
10     syscall
11     addi $t1, $t1, 1
12     j for_j
13 prox:
14     li $v0, 4
15     la $a0, nl
16     syscall
17     addi $t0, $t0, 1
18     j for_i
19 fim:
20     li $v0, 10
21     syscall
```

Equivalência C $\rightarrow$ MIPS

- `mat[i][j]` $\rightarrow$ cálculo manual de endereço
- `for` $\rightarrow$ labels + branches
- `printf` $\rightarrow$ `syscall`
- acesso à memória $\rightarrow$ `lw`

Funções em MIPS

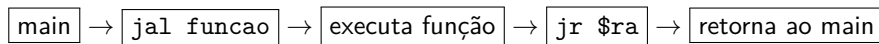
Funções permitem modularizar programas

- Dividem o programa em blocos reutilizáveis
- Facilitam manutenção e organização
- Permitem reutilização de código
- Funcionam de forma semelhante às funções em C

Principais elementos

Instrução	Função
jal	Chama uma função e salva automaticamente o endereço de retorno em \$ra.
jr \$ra	Retorna da função utilizando o endereço armazenado em \$ra.
\$a0-\$a3	Registradores usados para passagem de argumentos para funções.
\$v0	Registrador utilizado para armazenar o valor de retorno da função.

Fluxo de Execução de uma Função



O que acontece internamente?

- 1 O programa executa jal
- 2 O endereço da próxima instrução é salvo em \$ra
- 3 O processador salta para a função
- 4 A função executa seu código
- 5 jr \$ra retorna ao ponto original

Passagem de Parâmetros

Os primeiros argumentos da função são passados pelos registradores:

$\$a0, \$a1, \$a2, \$a3$

Exemplo em C

soma(10, 20)

Equivalente em MIPS

$\$a0 = 10$

$\$a1 = 20$

O resultado normalmente é retornado em:

$\$v0$

Registradores em Funções

Registrador	Uso
\$a0-\$a3	Passagem de argumentos
\$v0-\$v1	Valores de retorno
\$ra	Endereço de retorno da função
\$sp	Controle da pilha (stack)
\$t0-\$t9	Valores temporários
\$s0-\$s7	Valores preservados entre chamadas

Observação

Funções recursivas normalmente utilizam a pilha para salvar registradores e parâmetros.

Exemplo em C

```
1
2 #include <stdio.h>
3
4 int dobro(int x){
5
6     return x * 2;
7 }
8
9 int main(){
10
11     int y;
12
13     y = dobro(5);
14
15     printf("%d\n", y);
16
17     return 0;
18 }
```

Exemplo em MIPS

```
1  
2 .text  
3 .globl main  
4  
5 main:  
6  
7     # argumento  
8     li $a0, 5  
9  
10    # chama funcao  
11    jal dobro  
12  
13    # resultado em $v0  
14    move $t0, $v0  
15  
16    # imprime resultado  
17    li $v0, 1  
18    move $a0, $t0  
19    syscall  
20  
21    # encerra  
22    li $v0, 10  
23    syscall  
24  
25 dobro:  
26  
27     add $v0, $a0, $a0  
28  
29     jr $ra
```

Exemplo de Função

```
1 main:
2     li $a0, 5
3     jal dobro
4
5     li $v0, 10
6     syscall
7
8 dobro:
9     add $v0, $a0, $a0
10    jr $ra
```

Manipulação da Pilha

- A pilha cresce para baixo
- \$sp aponta para o topo
- Usada em chamadas de função
- Permite recursão

Salvando Registradores

```
1 addi $sp, $sp, -8
2 sw $ra, 4($sp)
3 sw $s0, 0($sp)
4
5 # codigo
6
7 lw $s0, 0($sp)
8 lw $ra, 4($sp)
9 addi $sp, $sp, 8
```

Mais de 4 Argumentos – Parte 1

Código em C

```
1
2 #include <stdio.h>
3
4 int soma(
5     int a,
6     int b,
7     int c,
8     int d,
9     int e,
10    int f
11 ){
12
13     return a+b+c+d+e+f;
14 }
15
16 int main(){
17
18     int x;
19
20     x = soma(
21         1,2,3,
22         4,5,6
23     );
24
25     printf("%d\n", x);
26
27     return 0;
28 }
```

Código MIPS – Parte 1

```
1
2 .text
3 .globl main
4
5 main:
6     # argumentos
7     li $a0, 1
8     li $a1, 2
9     li $a2, 3
10    li $a3, 4
11    # espaço stack
12    addi $sp,$sp,-8
13    # 5 argumento
14    li $t0, 5
15    sw $t0,0($sp)
16    # 6 argumento
17    li $t1, 6
18    sw $t1,4($sp)
19    jal soma
20    # limpa stack
21    addi $sp,$sp,8
```

Mais de 4 Argumentos – Parte 2

Mapeamento de Argumentos

Argumento	Local
1	\$a0
2	\$a1
3	\$a2
4	\$a3
5	stack
6	stack

Descrição

- Registradores possuem limite de 4 parâmetros
- Parâmetros extras são armazenados na pilha
- A função acessa os valores usando lw
- O resultado retorna em \$v0

Código MIPS – Parte 2

```
1
2     # imprime resultado
3     move $a0,$v0
4     li $v0,1
5     syscall
6     li $v0,10
7     syscall
8 soma:
9     # soma argumentos
10    add $t0,$a0,$a1
11    add $t0,$t0,$a2
12    add $t0,$t0,$a3
13    # lê stack
14    lw $t1,0($sp)
15    lw $t2,4($sp)
16    add $t0,$t0,$t1
17    add $t0,$t0,$t2
18    move $v0,$t0
19    jr $ra
```

Exemplo clássico:

$$F(n) = F(n - 1) + F(n - 2)$$

Cada chamada:

- salva registradores
- cria um frame na pilha
- retorna ao final da execução

Função Fibonacci – C x MIPS

Código em C

```
1
2 #include <stdio.h>
3
4 int fib(int n){
5
6     if(n <= 1)
7         return n;
8
9     return fib(n-1)
10        + fib(n-2);
11 }
12
13 int main(){
14
15     int x = 6;
16
17     int y = fib(x);
18
19     printf("%d\n", y);
20
21     return 0;
22 }
```

Sequência Fibonacci

0, 1, 1, 2, 3, 5, 8, ...

Código MIPS

```
1 .text
2 .globl main
3 main:
4     li $a0, 6
5     jal fib
6     move $a0, $v0
7     li $v0, 1
8     syscall
9     li $v0, 10
10    syscall
11 fib:
12    addi $sp,$sp,-12
13    sw $ra,8($sp)
14    sw $a0,4($sp)
15    ble $a0,1,base
16    addi $a0,$a0,-1
17    jal fib
18    sw $v0,0($sp)
19    lw $a0,4($sp)
20    addi $a0,$a0,-2
21    jal fib
22    lw $t0,0($sp)
23    add $v0,$v0,$t0
24    j fim
25 base:
26    move $v0,$a0
27 fim:
28    lw $ra,8($sp)
29    addi $sp,$sp,12
30    jr $ra
```

Alocação Dinâmica em MIPS

O que é alocação dinâmica?

Permite reservar memória durante a execução do programa.

Muito utilizada para:

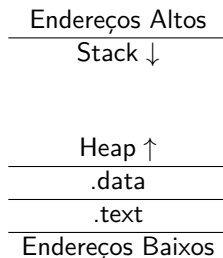
- vetores dinâmicos
- matrizes
- listas encadeadas
- pilhas e filas

Em MIPS utilizamos a syscall 9

```
sbrk(bytes)
```

Ela aumenta a região **heap** e retorna o endereço da memória alocada.

Heap e Stack



Características

- Heap cresce para cima
- Stack cresce para baixo
- A syscall 9 aloca memória na heap
- O endereço retornado normalmente é armazenado em um registrador

Funcionamento da Syscall 9

Passos para alocar memória

- 1 Colocar o código da syscall em:

$\$v0 = 9$

- 2 Informar quantidade de bytes em:

$\$a0$

- 3 Executar:

`syscall`

- 4 O endereço da memória alocada retorna em:

$\$v0$

Exemplo Para alocar um vetor com 10 inteiros:

$$10 \times 4 = 40 \text{ bytes}$$

Exemplo em C

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(){
5
6     int *vet;
7
8     vet = malloc(10 * sizeof(int));
9
10    vet[0] = 100;
11
12    printf("%d\n", vet[0]);
13
14    free(vet);
15
16    return 0;
17 }
```

Exemplo em MIPS

```
1 .text
2 .globl main
3 main:
4     # syscall 9
5     li $v0, 9
6     # 10 inteiros * 4 bytes
7     li $a0, 40
8     syscall
9     # endereço retornado
10    move $t0, $v0
11    # vet[0] = 100
12    li $t1, 100
13    sw $t1, 0($t0)
14    # imprime vet[0]
15    lw $a0, 0($t0)
16    li $v0, 1
17    syscall
18    # encerra
19    li $v0, 10
20    syscall
```

Explicação do Código

Registrador	Função
\$v0	Define a syscall e recebe o endereço retornado
\$a0	Quantidade de bytes a serem alocados
\$t0	Armazena o endereço base do vetor
\$t1	Valor armazenado no vetor

Observações

- Diferente da linguagem C, MIPS não possui malloc
- A syscall 9 é equivalente ao malloc
- O programador deve controlar manualmente os endereços
- Cada inteiro ocupa 4 bytes

Exemplo de Alocação

```
1 li $a0, 64
2 li $v0, 9
3 syscall
4
5 move $s0, $v0
```

Manipulação de Arquivos em MIPS

Syscalls para arquivos

Código	Operação	Entrada	Retorno
13	open	\$a0 = nome do arquivo \$a1 = modo de abertura \$a2 = permissões	\$v0 = descritor do arquivo
14	read	\$a0 = descritor \$a1 = buffer \$a2 = quantidade de bytes	\$v0 = bytes lidos
15	write	\$a0 = descritor \$a1 = buffer \$a2 = quantidade de bytes	\$v0 = bytes escritos
16	close	\$a0 = descritor	\$v0 = status

Observações

- O código da syscall deve ser colocado em:

Modos de Abertura de Arquivos

Valor em \$a1	Modo
0	Somente leitura
1	Somente escrita
9	Escrita com append

Exemplo

`$v0 = 13`

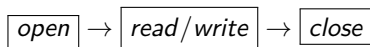
`$a0 = endereço do nome do arquivo`

`$a1 = 0`

`syscall`

O descritor retornado em `$v0` será usado nas operações de leitura e escrita.

Fluxo de Manipulação de Arquivos



Passos

- 1 Abrir o arquivo usando syscall 13
- 2 Guardar o descritor retornado
- 3 Ler ou escrever dados
- 4 Fechar o arquivo usando syscall 16

Importante

O descritor funciona como um identificador do arquivo aberto.

Leitura de Arquivo TXT – Parte 1

Código em C

```
1
2 #include <stdio.h>
3
4 int main(){
5     FILE *fp;
6     int num;
7     fp = fopen(
8         "dados.txt",
9         "r"
10    );
11    if(fp == NULL){
12
13        printf(
14            "Erro\n"
15        );
16        return 1;
17    }
18    while(
19        fscanf(
20            fp, "%d", &num
21        ) == 1
22    ){
23        printf(
24            "%d\n",
25            num
26        );
27    }
28    fclose(fp);
29    return 0;
30 }
```

Descrição

- O arquivo dados.txt é aberto em modo leitura
- fscanf lê inteiros do arquivo
- Cada número é impresso na tela
- O loop termina quando o fim do arquivo é alcançado
- fclose fecha o arquivo

Exemplo de arquivo

```
10
20
30
40
```

Leitura de Arquivo TXT – Parte 2

Código MIPS

```
1 .data
2
3 arquivo:
4 .asciiz "dados.txt"
5
6 buffer:
7 .space 128
8
9 .text
10 .globl main
11
12 main:
13     # open
14     li $v0,13
15     la $a0,arquivo
16     li $a1,0
17     li $a2,0
18     syscall
19     move $s0,$v0
20     # read
21     li $v0,14
22     move $a0,$s0
23     la $a1,buffer
24     li $a2,128
25     syscall
```

Código MIPS (continuação)

```
1     # print string
2     li $v0,4
3
4     la $a0,buffer
5     syscall
6     # close
7     li $v0,16
8     move $a0,$s0
9     syscall
10    # exit
11    li $v0,10
12    syscall
```

Descrição

- syscall 13: abre arquivo
- syscall 14: lê bytes do arquivo
- syscall 4: imprime conteúdo
- syscall 16: fecha arquivo

Leitura de Float em Arquivo – Parte 1

Código em C

```
1
2 #include <stdio.h>
3 int main(){
4
5     FILE *fp;
6     float x;
7     fp = fopen(
8         "dados.txt",
9         "r"
10    );
11    if(fp == NULL){
12
13        printf(
14            "Erro\n"
15        );
16
17        return 1;
18    }
19    while(
20        fscanf(
21            fp, "%f", &x
22        ) == 1
23    ){
24        printf(
25            "%.2f\n",
26            x
27        );
28    }
29    fclose(fp);
30    return 0;
```

Descrição

- fopen abre o arquivo em modo leitura
- fscanf lê números ponto flutuante
- printf imprime os valores
- fclose fecha o arquivo

Arquivo exemplo

1.5
2.7
3.14
4.8

Leitura de Float em Arquivo – Parte 2

Código MIPS

```
1 .data
2 arquivo:
3 .ascii "dados.txt"
4 buffer:
5 .space 128
6 .text
7 .globl main
8
9 main:
10     # open
11     li $v0,13
12     la $a0,arquivo
13     li $a1,0
14     li $a2,0
15     syscall
16     move $s0,$v0
17     # read
18     li $v0,14
19     move $a0,$s0
20     la $a1,buffer
21     li $a2,128
22     syscall
```

Código MIPS (continuação)

```
1     # print buffer
2     li $v0,4
3     la $a0,buffer
4     syscall
5     # close
6     li $v0,16
7     move $a0,$s0
8     syscall
9     # exit
10    li $v0,10
11    syscall
```

Observações

- syscall 14 lê bytes do arquivo
- O conteúdo é armazenado no buffer
- syscall 4 imprime o texto lido
- Conversão texto → float não é automática em MIPS

Aplicações:

- Blur
- Sharpen
- Conversão para grayscale
- Manipulação de imagens PGM

Kernel blur:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Depuração no MARS

- Execução passo a passo
- Breakpoints
- Visualização da pilha
- Visualização da memória
- Registradores em tempo real

- MIPS aproxima software e hardware
- Permite compreender execução de programas
- Fundamenta compiladores e arquitetura
- Excelente ferramenta didática